```
  def handleEvents(state: State, event: Event): State = event match {
    case FundsReserved(amount) =>
      State(state.balance - amount)          ❶
    case FundsAdded(amount) =>
      State(state.balance + amount)          ❷
    case FundsReservationDenied(_) =>
      state                                  ❸
  }}
  ...
}
```

❶ Reduces the balance

❷ Increases the balance

❸ Balance unchanged

This is the most basic implementation of a wallet. You could extend the use case by, for example, verifying with third parties that the request is allowed under gambling laws. Or you could link this wallet to a customer entity that contains more information about the user. But we'll leave it at this.

12.1.2 The market

This actor is responsible for holding the odds on a sporting event for a given bet. The winner of a soccer game, a player scoring a certain number of goals, and a horse finishing last in a race are all examples of markets, but this example betting house only models soccer games where the outcome can be a home win, an away win, or a draw. This sports event is modeled as a `Fixture` and the possible outcomes with `Odds`:

```
object Market {
  ...

  final case class Fixture(id: String, homeTeam: String, awayTeam: String)
      extends CborSerializable
```